arXiv:2305.05364v1 [cs.LG] 9 May 2023

Large Language Model Programs

Imanol Schlag [1] **Sainbayar Sukhbaatar** [2] **Asli Celikyilmaz** [2] **Wen-tau Yih** [2] **Jason Weston** [2]
Jürgen Schmidhuber [1 3] **Xian Li** [2]

Abstract

In recent years, large pre-trained language models (LLMs) have demonstrated the ability to follow instructions and perform novel tasks from a few examples. The possibility to parameterise an LLM through such in-context examples widens their capability at a much lower cost than finetuning. We extend this line of reasoning and present a method which further expands the capabilities of an LLM by embedding it within an algorithm or program. To demonstrate the benefits of this approach, we present an illustrative example of evidence-supported question-answering. We obtain a 6.4% improvement over the chain of thought baseline through a more algorithmic approach without any finetuning. Furthermore, we highlight recent work from this perspective and discuss the advantages and disadvantages in comparison to the standard approaches.

1. Introduction

Scaling language models to hundreds of billions of parameters (LLMs) and training them on terabytes of text data has led to state-of-the-art performance on a large variety of natural language processing tasks. Additionally, it has also led to the emergent ability to learn a new skill merely from instructions or a few examples, as seen in GPT-3 (Brown et al., 2020). Despite this, LLMs struggle to display algorithmic abilities, such as sorting or searching, even when finetuned on traces of such (Anil et al., 2022; Valmeekam et al., 2022; Liu et al., 2023; Deletang et al., 2023).

Finetuning a model on human traces (i.e. imitation learning) is a common strategy to infuse a model with complex behaviours. For example, in the context of LLMs, recent work trains on expert demonstrations of interacting with a browser (Nakano et al., 2021) or a webshop (Yao et al., 2022) in order to improve their respective tasks. However, there are reasons to question the correctness of the algorithms learned from such examples. First, the decoder-only Transformer language model (Vaswani et al., 2017) is not computationally universal because it can only condition on a finite length input (Fan et al., 2021). Second, previous work demonstrated that the decoder-only architecture struggles to learn even simple programs through gradient descent that would generalise out of distribution (Anil et al., 2022; Deletang et al., 2023).

Besides finetuning, in-context learning may be leveraged to improve a model's ability to execute algorithms. E.g., recently Zhou et al. (2023b) introduce a prompt construction which improves the LLM's ability to perform arithmetic algorithms. A key characteristic of such approaches is the use of a single call to the model and the use of a single prompt. As a consequence, the method is limited by the size of the input and the necessity to prompt engineer multiple algorithm steps within one prompt. The authors report that this can lead to interference between steps which hurts the final performance.

As an alternative, we propose embedding LLMs into a program or algorithm. Crucially, instead of the LLM being responsible for maintaining the current state of the program (i.e. its context), the LLM, for each step of the program, is only presented with a step-specific prompt and context. Hiding information which is irrelevant to the current step allows us to focus on isolated subproblems whose results are further combined in future calls to the LLM. This intuitive approach allows us to extend the ability of an LLM to more complex tasks which are currently too difficult either because of a lack of ability or an architectural constraint such as an insufficiently large context. Concurrent work proves that embedding an LLM within a recurrent algorithm which enables the interaction with an external memory component makes the system computationally universal (Schuurmans, 2023).

Both approaches, embedding and finetuning, incorporate domain-specific information and thus lead to a more specialised model. Embedding the model in an algorithm, or *program with an LLM*, forces the LLM to follow a high-level procedure (whatever it may be) which may limit its flexibility and requires a high-level understanding of the

[1]The Swiss AI Lab IDSIA, SUPSI & USI [2]Meta AI [3]King Abdullah University of Science and Technology. Correspondence to: Imanol Schlag <imanol@idsia.ch>.

Preprint. Under review. Copyright 2023 by the author(s).

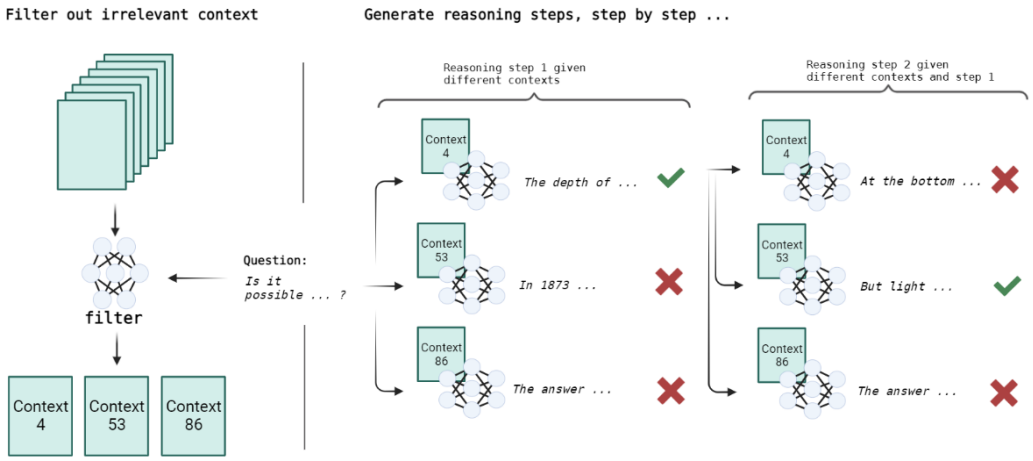

Figure 1. An illustration of our two-part LLM program example. The first part (left) of our program filters out irrelevant paragraphs from a large set. This is achieved using an LLM and the current question. The second part (right) depicts the generation of the individual reasoning steps until they result in an answer. The tree search is over the choice of paragraph used within the context when generating a single step in the reasoning chain. We expand the reasoning chain which ranks the highest as described in Section 3.1.2.

correct procedure for a given task. On the other hand, fine-tuning requires the recognition, and possibly generation, of domain-specific training data which requires a similar level of understanding. We argue that there are situations where it may be more promising to embed the LLM in a program explicitly instead of training on traces of a desired program. Due to the difficulty and cost of training LLMs we have already seen problem-specific LLM programs with no or very targeted training lead to better performance on specific problems such as reasoning (Kazemi et al., 2022), question-answering (Zhou et al., 2023a), text-summarization (Wu et al., 2021a), text-generation (Yang et al., 2022b), among others (Zeng et al., 2023; Xu et al., 2021; Karpas et al., 2022).

In this work, we present the advantages and disadvantages of programming with LLMs and present a general approach which we call a *Large Language Model Program* (Section 2). In Section 3, we then present an LLM program example for evidence-supported question answering. In order to accurately answer a question, the program first extracts the important facts from the knowledge sources and seamlessly incorporates them into the reasoning of its response. A simple illustration of the program is given in Figure 1. We then go on to describe how our general approach can be applied to various other settings, described in Section 4.

2. Limitations of LLMs and the Benefit of Programming With Them

LLMs are a product of three necessary ingredients: massive parallel compute, large amounts of data, and a highly parallelisable language modelling architecture. To this date, the architecture deployed by all LLMs of 175B parameters or more is the decoder-only Transformer (Vaswani et al., 2017). The ability to parallelise allows for efficient training of large models and the use of large amounts of data prevents it from overfitting. In fact, when training LLMs they rarely see any training data twice.

LLMs which were trained on internet-scale data are impressive text generators that can produce plausible paragraphs in the form of short stories, reviews, summaries, poems, etc. Such *raw* LLMs are trained on data that includes high-quality texts such as those from Wikipedia or published books but also subjectively low-quality texts with toxic and biased content which then leads to LLMs that are to a certain degree toxic and biased themselves (Dev et al., 2020; Sheng et al., 2021).

Another drawback of training on random internet text is that such raw LLMs also struggle to be conversational, follow instructions, use tools, or interact with an environment. Although finetuning on examples of such behaviour has been shown to improve the model's ability (Ouyang et al., 2022; Wei et al., 2022a; Gupta et al., 2022), such an approach is difficult to scale due to the lack of such data and may not generalise systematically due to the possibility of leveraging algorithmic short-cuts (Liu et al., 2023).

Another limitation of LLMs arises from their decoder-only Transformer architecture, which has a finite context that restricts their capability to only process information within the predefined context. While this architecture may be advantageous for tasks like translation or language modelling, it struggles to learn certain classes of algorithms accurately, which can affect its ability to generalize beyond its training

distribution (Csordás et al., 2021).

Such issues are likely going to be present also at a larger scale if an LLM is, either implicitly or explicitly, finetuned on traces of algorithms. One example is presented by Anil et al. (2022) who finetune LLMs of up to 64B parameters on a variation of parity and report large drops in performance for just slightly longer (or shorter) sequences than it was trained on. It is worthwhile to mention that it has long been known that recurrent neural networks, such as the Long Short-Term Memory (LSTM), can generalise perfectly on parity and other counter-based context-sensitive languages (Hochreiter & Schmidhuber, 1997; Gers & Schmidhuber, 2001; Deletang et al., 2023).

To overcome such limitations, we propose *LLM programs*, a general approach to enhance the capability of an LLM-based system. Using an LLM program, we recognise the limitation of the LLM as a general agent. Instead of further training the model, we recursively deconstruct the expected behaviour into simpler steps that the LLM can perform to a sufficient degree. These individual steps are then strung together by a classic computer program (such as Python) that parses the outputs of previous steps, uses control flow, and augments the prompts of succeeding steps.

One important distinction of LLM programs from previous work that relies on external systems like calculators (Thoppilan et al., 2022) is that the current state of the program is not maintained by the LLM but by the program in which the LLM is embedded. Consequently, every call to the LLM only includes the information that is necessary for the particular step. There are several advantages to LLM programs:

- Embedding an LLM in a program can significantly expand the theoretical and practical capabilities of the system with no or little finetuning and can help the system generalise more systematically.
- LLM programs incorporate high-level algorithmic information by breaking down a complex task into multiple simpler steps with little or no training. In contrast, learning to perform a complex task from examples alone may require a large amount of compute and high-quality data.
- Understanding the problem decomposition allows for more fine-grained input and output specifications for each subproblem or step of the program. These specifications allow for a more manageable and systematic evaluation of the LLM's performance by testing the model on each subproblem in isolation, either through a test set or manual inspection.
- An LLM program performs many queries to the model instead of just one. Each query may contain a step-specific prompt that improves the model's performance for that particular step. Prompt engineering in a setting with a single query to the LLM may find it much more challenging to provide an effective prompt because examples or descriptions for different steps may interfere with each other and the prompt overall will take up more space within its already limited context.

In summary, LLM programs offer a promising way to address the limitations of LLMs and expand their practical utility in various domains. In the next section, we will take the reader through an example of an LLM program for the purpose of question-answering.

3. An LLM Program for Question Answering

Motivating example. Imagine using an LLM as an interface for a company's website. The LLM was not trained on that website, which means that it would not be able to answer questions about the company's newest products, such as its features, specifications, or availability, or about the opening hours of a new store. In this example of reading comprehension, an LLM would perform poorly because it has not been trained on the company's website nor can it fit the entire website into its context. Finetuning the LLM may not be practical, as training these models is costly and websites often undergo changes (e.g. a product may unexpectedly become unavailable) which would require the model to be constantly retrained. Additionally, it is unclear to which extent the finetuned model would be faithful to the facts of the finetuning data.

A more practical approach is to use a frozen LLM that has exclusive access to the latest version of the website where each page represents one document. Since we cannot load all documents at once, we may construct a program to carefully select the documents necessary to answer a user's question.

The dataset. We construct a dataset to develop an LLM program which can answer questions by leveraging many additional knowledge sources. To this end, we take advantage of the StrategyQA dataset (Geva et al., 2021). The StrategyQA dataset is a binary classification benchmark for implicit multi-step question-answering. The reasoning traces in StrategyQA are more difficult than previous question-answering datasets due to different forms of question decomposition and the need for detailed knowledge from different domains. Consider e.g. the question *Can sunlight travel to the deepest part of the Black Sea?*. In order to answer this question the LLM needs to know that the deepest part of the Black Sea is about 2k meters whereas sunlight only penetrates water up to about 1k meters. Having that information, inferring the answer becomes significantly easier for an LLM . See our experiments in Table 2 where the

LLM performs best when it has all the necessary facts in its context.

We build a question-answering dataset from StrategyQA where every question is supported with a large number of knowledge sources of which only a few are actually relevant to the current question. Most of them will be randomly sampled from other questions and are thus very unlikely to contain any useful information. Crucially, even just a handful of those paragraphs would be too long for the context length of current LLMs.

The StrategyQA dataset in total consists of 2780 questions annotated with their decomposition and per-step evidence paragraphs sourced from Wikipedia. About 918 of the questions in StrategyQA come with evidence paragraphs that support all reasoning steps. We will limit ourselves to this subset to ensure that the additional knowledge sources actually contain all information necessary to infer the answer. Otherwise, it would require the LLM to have learned the necessary facts from its pre-training data, which we do not assume to be the case.

Previous work reported mixed performances for question-answering on the regular StrategyQA benchmark (Chowdhery et al., 2022). This might be because the facts used for individual reasoning steps are very rare and unlikely to be remembered or because the reasoning is too complex. While improved reasoning methods (such as chain of thought (Wei et al., 2022b; Nye et al., 2022; Kojima et al., 2022; Wang et al., 2023)) have shown little performance gain on this dataset with models of 175B parameters (Srivastava et al., 2022; Taylor et al., 2022), larger models and models which have been trained longer do seem to improve (Chowdhery et al., 2022; Hoffmann et al., 2022). This may indicate that the lack of knowledge is more problematic on this dataset than the ability to reason.

The LLM. In our experiments, we use OPT-175B (Zhang et al., 2022), a raw language model of 175B parameters trained on 300B tokens from a large mix of news articles, books, Pushshift.io Reddit previously compiled by a third party, and the Pile (Gao et al., 2020). Because it has not been finetuned on high-quality instructional or conversational data, we classify it as a raw language model. As reported by Zhang et al. (2022), we find OPT-175B struggles to follow instructions. Nevertheless, we demonstrate that we can achieve more complex behaviour by embedding OPT within a program.

3.1. Developing the LLM Program

For the problem of evidence-supported question-answering, we decompose the problem into a filtering part and a tree search part. Given the question, the filtering part will loop over the provided paragraphs and select the most relevant ones. In the second part, we will search over reasoning chains by generating one reasoning step at a time. When generating a reasoning step we can choose which one of the filtered paragraphs we want to condition on. Similar to a tree search, we rank the reasoning chains according to our ranking metric and expand the highest-ranked one by generating n possible continuations given n evidence paragraphs. This process repeats until it produced an answer or until it reached the maximum number of steps at which point it will force the LLM to answer the question by evaluating the negative log-likelihood (NLL) of a fixed yes or no answer.

An example trace. When answering a question like *Could the members of The Police perform lawful arrests?* we use the LLM to select a few paragraphs from a much bigger collection of which many are not relevant to this particular question. For this example, selecting the top 10 paragraphs according to OPT is sufficient to capture all the relevant information to successfully reason about the answer. To find a strong reasoning sequence, we generate each step using a different paragraph as context. The tree search is over the choice of paragraph to be used as context for a particular step. For the question about the band *The Police* the highest ranked step has a paragraph in its context which is about the members of the band. Given that context, the model generates the first step *The members of The Police are Sting, Stewart Copeland, Andy Summers, and Henry Padovani*. Following the first step, the highest-ranked step is conditioned on another paragraph with more details about the band The Police in general. This results in the second step which says *The members of The Police are not police officers.* After that step, the model's highest-ranked next step is conditioned on a paragraph which explains that only police offers are allowed to arrest people. From that the generated next step says *Thus, the members of The Police could not perform lawful arrests.*. Finally, the model generates the last step *Thus, the answer is no.*, which is recognised as an answer and evaluated to be correct.

In the following two subsections, we go into more detail about each part of our LLM program.

3.1.1. Evidence Filtering Experiments

This section describes our approaches to the paragraph filtering part of our program and their performance. Developing and evaluating each call to the LLM in isolation allows us to experiment, improve, and analyse the overall performance in a systematic way. Our first attempt will rely on few-shot examples and prompt-engineering the model to classify one paragraph given the question.

Blackbox prompting. Our first approach centres on few-shot prompting, using OPT to output a yes or no answer when asked whether a certain paragraph is relevant to a spe-

cific question. This binary classification dataset comprises 300 samples randomly selected from the StrategyQA data. Each sample includes a question and an associated paragraph, and the objective is to classify whether the paragraph is relevant to the question.

We carefully designed the dataset for this subproblem so that half of the questions are paired with paragraphs randomly selected from a list of evidence paragraphs, while the other half are paired with unrelated paragraphs from other questions. This ensures an equal distribution of relevant and irrelevant paragraphs which puts the random baseline at 50%. Our experiments in Table 1 indicate that neither OPT nor InstructGPT (Ouyang et al., 2022) achieves satisfactory performance in classifying a single paragraph. Even *Tk-Instruct* (Wang et al., 2022), a model which has been trained on a large variety of distinct and expert-written tasks, fails to reliably recognise relevant paragraphs.

This is surprising since the classification task seems simple for humans: it is often inconceivable how a randomly sampled paragraph could be relevant to the question. Furthermore, evidence paragraphs often share words or phrases or are semantically relevant to the question. Because of the outcome of our isolated experiment, we decide to explore an alternative approach.

Table 1. Top-1 binary classification accuracies for a single paragraph using prompt engineering techniques. Random guessing is at 50%. Note that using this method to classify from a large set of paragraphs will significantly reduce the accuracy.

METHOD	ACCURACY
OPT-175B FEW-SHOT PROMPT	53.33%
TEXT-DAVINCI-002 (INSTRUCTGPT)	55.67%
OPT-175B FEW-SHOT + CHAIN OF THOUGHT	56.00%
TK-INSTRUCT 11B	61.60%

Likelihood evaluation. Our second approach does not treat the LLM as a black box anymore but instead directly uses the average negative log-likelihood (NLL) of the question when following each paragraph to create a ranking of all paragraphs. To evaluate this subproblem in isolation, we constructed a new dataset. Each question now consists of 100 paragraphs where one is sampled from the list of evidence paragraphs of that question and the other 99 are sampled from other questions. In this setting, random guessing only achieves 1% accuracy. Notice that 100 paragraphs clearly exceed the context length of OPT. We present the pseudo-code in Algorithm 1 (see Appendix).

We achieve good results without a description or few-shot examples. We plot accuracy over top-n in Figure 3 and the average likelihood of each paragraph given each question in Figure 2. Top-1 accuracy is 0.79 and top-5 is already at

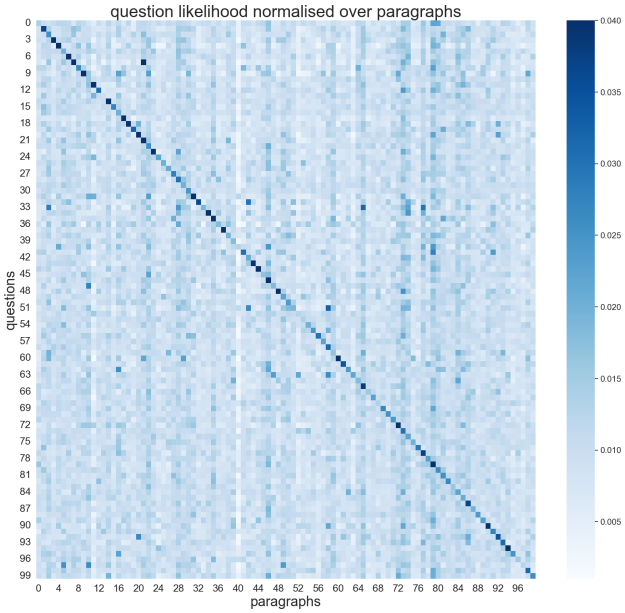

Figure 2. Average likelihood of each question (row) given a paragraph (column) normalised over paragraphs.

0.93 which shows that the ranking approach is vastly more effective than the blackbox prompting approach despite the simplicity of the task and the instruction-finetuning of some of the models. It also serves as an example of the advantage of embedding an LLM within a program because the LLM does not have access to the NLL of its own outputs and, thus, this superior solution would have not been possible otherwise.

3.1.2. TREE SEARCH EXPERIMENT

In the previous subsection, we presented the filtering part of our program: measuring the relevancy of each paragraph, ordering them, and selecting the top n. It trivially generalises in a systematic way to more paragraphs since the processing of the list of paragraphs is done by a Python for-loop and doesn't solely rely on an LLM's ability to implement it reliably. This is especially powerful if we know what the optimal algorithm or behaviour should be and if we want to have control over it.

In this section, we present a more systematic search over reasoning steps to ensure generalisation where we consider it most crucial. As in the filtering part, we develop and test the tree search in isolation using a dedicated dataset.

Raw LLMs do not have access to an external system when deducing answers to difficult questions. This means that the model can only rely on the noisy and inevitably incomplete and outdated knowledge that has been stored in its weights. The questions in the StrategyQA dataset seem to often require knowledge that is not present in LLMs. For this reason,

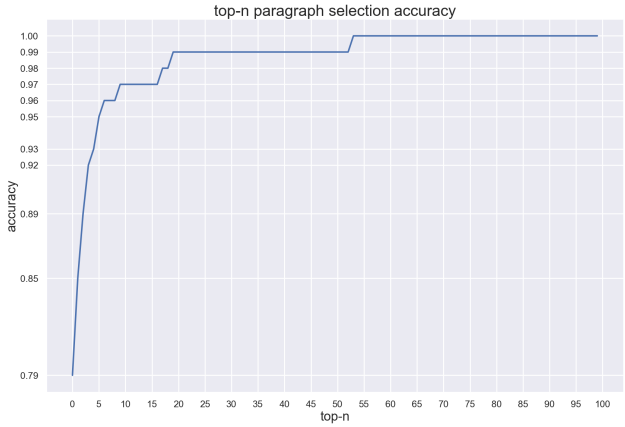

Figure 3. Top-n accuracy of selecting the true evidence paragraph from our likelihood ranking of 100 paragraphs. The random guessing baseline for top-1 is 1%. The ranking approach significantly outperforms the in-context approach from Table 1.

we introduce an evidence-supported chain of thought where the reasoning chain is generated over multiple steps and each step has a different support paragraph within its context. This allows the model to use information from the paragraph to generate reasoning steps it would otherwise be unable to produce. For StrategyQA most questions rely on reasoning steps which draw from different knowledge domains. In order to find the most valuable paragraph for each reasoning step, we perform a tree search where the root node is the question and each level of the tree spans over the set of possible evidence paragraphs used as context to generate the next step. Every path through the tree is a (possibly incomplete) reasoning chain. Searching over all possible reasoning chains is infeasible which is why we rank all reasoning chains and continually expand the highest-ranked chain. We present pseudo-code in Algorithm 2 (see Appendix).

We explore two ranking strategies. The first is the average NLL of the chain so far (where the average NLL of each step S is computed when conditioning on its respective paragraph). With this approach, the model will expand the reasoning chain which has been the most likely so far. This approach works reasonably well but can lead to issues. We find that if an LLM directly copies entire phrases from the evidence paragraph P, they will be given a very low NLL. This can lead to repetitions or ongoing deductive steps which quickly end up in arguing about things that are irrelevant to the question Q. Our second ranking strategy is an approach which tries to mitigate such issues by ranking the generated reasoning steps by their average NLL *difference*: with and without the paragraph (Δ_P), and with and without the question (Δ_Q). Those length-normalised differences, Δ_P and Δ_Q , allow us to select reasoning chains which leverage the provided context (which reduces hallucinations) but remain on-topic (which reduces divergent reasoning).

$$\text{nll}(x) = -\log(x)/\text{len}(x) \quad (1)$$

$$\Delta_\text{P} = \text{nll}(p(S|Q,P)) - \text{nll}(p(S|Q)) \quad (2)$$

$$\Delta_\text{Q} = \text{nll}(p(S|Q,P)) - \text{nll}(p(S|P)) \quad (3)$$

Generated reasoning steps with a negative Δ_P rely more strongly on the paragraph and are thus more likely to incorporate information from it. Generated reasoning steps with a negative Δ_Q rely more strongly on the question which leads to less divergence and favours steps which remain on topic. We find that steps conditioned on paragraphs where $\Delta_\text{P} + \Delta_\text{Q}$ is the lowest leads to better reasoning chains and improves accuracy.

To evaluate the tree search in isolation, we assume that the evidence paragraphs contain information about all reasoning steps necessary to answer the question. This is not the case for all StrategyQA questions which is why we limit ourselves to the 918 StrategyQA questions which are fully supported.

Note that this technique is a variation of beam search (Medress et al., 1977), i.e. a heuristic search algorithm which explores a graph by expanding the highest-ranking node in a limited set. It is quite different from the usual use of beam search for generating text with language models where in order to generate the next token we select the k tokens with the highest probability given the same context. Instead, we always generate the most likely tokens but search over the limited set of k paragraphs to be used as context.

Our experimental results in Table 2 demonstrate that the model's reasoning ability has improved over the OPT chain of thought baseline. To better highlight the advantage of our tree search we include a baseline where we add the golden facts to the context of OPT using a custom prompt. Golden facts are statements provided by the authors of StrategyQA which should contain the factual knowledge necessary to deduce the correct answer. With the golden facts in its context, OPT achieves about 81.2%. This can be seen as a performance upper bound for the OPT model since it represents the setting where the model has all the necessary facts clearly within its context.

4. Further LLM Program Examples

Training an LLM is a costly and complex engineering challenge. For this reason, training and research on such models is mostly done in well-funded industrial labs. However, in recent years, GPT-3 (Brown et al., 2020) has become easily accessible to the public through its API (Brockman et al., 2018) and the weights of LLMs from the OPT (Zhang et al., 2022) and BLOOM (Scao et al., 2022) family have been made publicly available, sparking a flurry of LLM research.

Table 2. Binary classification accuracy on the StrategyQA subset with fully supported evidence. CoT stands for chain of thought. Golden facts are the facts necessary to answer the question according to the StrategyQA dataset.

METHOD	ACCURACY
OPT-175B, FEW-SHOT, NO-COT	50.33%
OPT-175B, FEW-SHOT, WITH-COT	60.11%
OPT-175B, FEW-SHOT, WITH-TREE-SEARCH, NLL RANKING	65.98%
OPT-175B, FEW-SHOT, WITH-TREE-SEARCH, DELTA RANKING	66.41%
OPT-175B, FEW-SHOT, WITH-GOLDEN-FACTS	81.27%
OPT-175B, FEW-SHOT, WITH-GOLDEN-FACTS, WITH-COT	81.12%

Some recent and concurrent work implicitly follows the presented LLM Program approach. Such work often has the characteristic that it describes stages or an algorithm and parameterises different steps with different prompts. In this Section, we'll go through all recent and concurrent work that uses this emerging methodology.

Creswell & Shanahan (2022) decompose the inference of an answer into a recurrent algorithm which consists of a selection and inference step, as well as a halting criterion. Each step then consists of a finetuned LLM which only has access to the information necessary in order to prevent the model from learning shortcuts, such as directly predicting the answer from the question instead of performing a deductive step. The final system's capability of finding proofs is increased by over 20% in comparison with baselines.

Yang et al. (2022c) present a modular approach to extract rules from natural language facts. To achieve this they use an LLM to generate many candidate rules based on input facts and the desired rule template. In four further steps, the same LLM with a different prompt is used to filter out rules which do not meet the necessary requirements for induction (e.g. triviality). The authors present experimental results for the entire system as well as each module in isolation which indicate the benefits of their 5-step program.

In a similar vein, various works propose the use of *verifiers* to improve the quality of the generated samples through some form of repeated verification (Cobbe et al., 2021). For example, Saparov & He (2023) leverages diverse prompts and verifies each individual step in a recurrent fashion, resulting in significant gains in accuracy without any finetuning.

Kim et al. (2021) present a 3-step program to identify unverifiable presuppositions in the question-answering setting. E.g., the question *Which linguist invented the lightbulb?* contains the false presupposition that a linguist has invented the lightbulb. The authors report that false presuppositions can make up a substantial share of the unanswerable questions in the popular Natural Questions (Kwiatkowski et al., 2019) dataset. Given a question, their program first generates presuppositions, verifies them, and finally, generates an explanation. The authors test different strategies where they use neural and rule-based models for different steps, testing each in isolation.

Current approaches to reasoning with language models are forward chaining, i.e. they start of with certain facts and search in the space of rules until they find a goal statement. This also includes our own method presented in Section 3.1.2, but Kazemi et al. (2022) argue that backwards chaining, i.e. decomposing the goal statement until the subgoals can be proven from the facts, is heavily favoured in the classic automated proof-finding literature. For this reason, they implement a backwards-chaining using four modules: fact check, rule selection, goal decomposition, and sign agreement. Each of the modules is implemented using a pre-trained LLM with a custom prompt and the modules are subroutines of the *LAMBADA* program. Thanks to the ability of the LLM, the modules remain relatively high-level which results in a relatively simple program with two nested for-loops. The LAMBADA approach significantly outperforms other methods such as chain of thought (Wei et al., 2022b) and selection-inference (Creswell & Shanahan, 2022).

Another related line of works first decomposes the original question into simpler subquestions which should be easier to answer. With the answers to the subquestions, a final module answers the original question. Patel et al. (2022) use questions decomposed by humans which significantly improves performance. Zhou et al. (2023a) use an LLM to automatically decompose questions. Their empirical results indicate that they significantly outperform chain of thought prompting. However, different from previous approaches, the authors perform a question decomposition within a single call to an LLM instead of implementing a recurrent algorithm. As a result, their approaches may be more prone to mistakes on more complex examples which are out of distribution since the output specification of such a compact step includes many more possibilities than in a more broken down program like LAMBADA. For further decomposition examples see e.g. Perez et al. (2020); Yang et al. (2022a).

Regular LLMs are isolated systems that can only access the information that has been transferred into their weights. As we argued in Section 2, such systems may lack the neces-

sary information to perform certain tasks. Leveraging an additional, possibly non-neural, system could be a viable alternative to an increase in model scale. E.g., Lazaridou et al. (2023) generate a google search query whose result will be added as context to the question-answering prompt. The improvements in factuality and accuracy indicate the benefits of "inference-type" interventions, i.e. embedding the model within a simple program which during inference allows the system to leverage results from a classic document retrieval system without the need for a dedicated retrieval-augmented model.

Interestingly, an LLM is not always able to use the knowledge that it has stored in its weights. This is shown by Liu et al. (2022b) who demonstrate a simple 2-step program which first generates a question-specific set of facts before answering the question can result in better performance (for similar methods see Paranjape et al. (2021); Li et al. (2022)). Liu et al. (2022a) further improve such knowledge extraction using reinforcement learning based on increased question-answering performance. See also the work of Cohen et al. (2023) on extracting a knowledge base from a language model using a multi-step program.

Apart from question-answering, we also see the emergents of LLM programs for generative tasks. Yang et al. (2022b) recursively prompt an LLM with the story plan and the current story state to generate coherent stories of up to 2500 words. Similarly, Wu et al. (2021b) summarise entire books by recursively summarising previous summaries of fixed-size chunks. Both are great examples of recursive programs which overcome the finite-context limitation of the decoder-only Transformer.

In another line of work, LLMs are embedded in an ongoing model-environment loop to plan and suggest actions to a robot given high-level goals (Ahn et al., 2022; Huang et al., 2022). However, current approaches give all context to the LLM and do not have a mechanism to update an external memory. As a result, the input and output specification of the model is likely to exceed the current capabilities of LLMs for certain extreme examples.

A rapidly growing body of work further generalises the notion of the environment which the LLM interacts with. These approaches combine the strength of an LLM with the strength of another connectionist or classic system. Examples of such include the LLMs which repeatedly generate short executable programs as intermediate reasoning steps (Gao et al., 2022), which generate chess commentaries with the help of symbolic reasoning engines (Lee et al., 2022), or which use a variety of other tools such as a calculator (Karpas et al., 2022).

In the future, we will likely see more elaborate algorithms being developed which embed several neural and symbolic modules to solve increasingly complex tasks. To support such projects, new libraries have been recently created. LangChain (Chase, 2023) is a library which supports developers to combine LLMs with other sources of computation. GPT-Index (Liu, 2022) is a collection of data structures designed to make it easier to use external knowledge bases with LLMs. Reppert et al. (2023) present ICE, an open-source tool for visualizing the execution traces of LM programs.

Recent work also studied the benefits of LLM programs. Wu et al. (2022b) study the user needs when authoring their own LLM programs and present PromptChainer to support building prototypes for applications. Wu et al. (2022a) present an interactive "chain authoring system" and study how users modify them to improve performance but also user satisfaction.

General purpose programs. Another interesting line of research is going to centre around task-*independent* LLM programs which, so far, have received much less attention. Shuster et al. (2022a) present a system which augments a conversational agent with a knowledge retrieval step before responding to the user. Similarly, Shuster et al. (2022b) present BlenderBot 3, a conversational bot based on an LLM but which is augmented with a classic long-term storage such that it can remember information about the user across several sessions (which e.g. ChatGPT (Schulman et al., 2022) cannot). Another approach is presented by Dalvi et al. (2022), who use a simple program in form of a dialogue tree to learn facts from the user. Facts are stored in a dynamic memory which the LLM also updates. Crucially, the program includes an interaction loop with the user which allows the user to correct the model which then updates its dynamic memory with new or corrected facts. Hence, this LLM program may be considered an example of an LLM program which implements a crude learning algorithm. This opens up an exciting direction for future research which can benefit from symbolic learning algorithms such as e.g. the Optimal Ordered Problem Solver or the Gödel Machine (Schmidhuber, 2004; 2007).

5. Discussion and Related Work

The presented method of programming with pre-trained models stands in opposition to the common deep-learning philosophy of training a single omniferous black-box model which only has to be scaled in terms of parameter count and training data. Although central to the current success of LLMs, such an approach comes with several drawbacks:

- The computation of deep connectionist models, such as a large pre-trained Transformer language model, is notoriously difficult to interpret (although there is on-

going research, see e.g. Elhage et al. (2021)). Breaking a problem down into multiple steps can not just improve performance but also increase the interpretability of its inference.

- LLMs are trained on large amounts of text which can contain toxic, biased, or otherwise undesired content. As a result, an LLM may also output undesired text. The language model itself does not provide any safety mechanisms to prevent such outputs. Embedding an LLM within a program is a simple and effective way to include a safety mechanism which e.g. filters out unwanted LLM responses.

- Tuning an LLM on expert trajectories of complex behaviour requires large amounts of high-quality and behaviour-specific data which is difficult and expensive to acquire. Breaking the problem into subproblems may allow the identification of specific lower-level capabilities that are missing. Focusing the data collection on such *blindspots* is potentially a faster and more efficient approach and lower-level capabilities may be useful for a wide range of problems.

- It is difficult to give any guarantees for neural models due to the lack of interpretability. Furthermore, it is well-known that neural networks struggle to generalise out of distribution. However, embedding one or several connectionist modules with a program allows us to give some trivial generalisation guarantees that do not hold otherwise. E.g., the filtering and search aspect of our program, as presented in Section 2, generalises trivially to a larger number of paragraphs.

- So far, all LLMs of 100B parameters or more are variations of the Transformer architecture. They thus inherit its limitations, such as a finite context of a few thousand tokens (Hutchins et al., 2022). Embedding an LLM within a task-independent program that is responsible to select and load relevant documents into context or which summarises past text may serve as a way of overcoming such limitations.

We believe that the method of programming with LLMs as presented in this work can mitigate many drawbacks in settings where the expected processing of a query is well understood. Many recent papers have demonstrated the benefits of such an approach (see the previous Section 4), but significantly fewer have taken a higher-level view. Here we mention some recent or concurrent works which are related to our general perspective.

Concurrent to our work, Khot et al. (2023) present *Decomposed Prompting*, a framework for decomposing tasks into subtasks which can be solved in isolation by LLMs using a subtask-specific prompt. The authors demonstrate better out-of-distribution performance on simple string manipulation and question-answering tasks.

In earlier work, Dohan et al. (2022) have presented a related perspective which describes compositions of pre-trained models as probabilistic programs. In so-called *language model cascades*, LLMs are random variables of type string within a probabilistic program. With such a perspective the authors present a unifying view of existing algorithms such as chain of thought (Wei et al., 2022b), scratchpads (Nye et al., 2022), or STaR (Zelikman et al., 2022), which complements our more algorithmic perspective.

Similarly, Creswell & Shanahan (2022) briefly describe their approach as *algorithmic prompting* where the response of a language model given the first prompt is integrated into future prompts. The authors argue that such prompt manipulations and constructions can be composed into entire algorithms to achieve more sophisticated behaviour. In follow-up work, Shanahan (2022) argues that such an approach is necessary to build a trustworthy reasoning system.

Zeng et al. (2023) propose a modular framework with multiple pre-trained models which are composed to capture new multimodal capabilities without the need for end-to-end training or finetuning.

A different but related approach describes a looped transformer model as *programmable computers* (Giannou et al., 2023). Through a simple loop they iteratively call a transformer model with a scratchpad, memory, and instructions, analogous to computer programs.

Our approach is inspired by the "learning to think" report (Schmidhuber, 2015) (Sec. 5.3) where a controller network C learns to send sequences of activations into another network M after which C reads M's activations in order to exploit its knowledge. Our program, however, is not a trained neural network but an explicit algorithm.

6. Conclusion

We have presented LLM programs, the emerging methodology of embedding pre-trained connectionist models, such as large language models, in a classic program to carry out more complex tasks. It is central to this method to decompose the main problem recursively into subproblems until they can be solved by a single query to the model. With more fine-grained input and output specifications for each subproblem, the model's capabilities can be developed and tested in isolation. We describe an example of this method in the setting of evidence-supported question-answering and demonstrate an improvement in performance without any finetuning. We also list the advantages and disadvantages and highlight recent works from this perspective.

References

Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., et al. Do as i can, not as i say: Grounding language in robotic affordances. In *6th Annual Conference on Robot Learning*, 2022. URL https://openreview.net/forum?id=bdHkMjBJG_w.

Anil, C., Wu, Y., Andreassen, A. J., Lewkowycz, A., Misra, V., Ramasesh, V. V., Slone, A., Gur-Ari, G., Dyer, E., and Neyshabur, B. Exploring length generalization in large language models. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=zSkYVeX7bC4.

Brockman, G., Murati, M., and Welinder, P., Sept 2018. URL https://openai.com/blog/openai-api/.

Brown, T. B. et al. Language models are few-shot learners. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, Virtual only, December 2020.

Chase, H. Langchain. https://github.com/hwchase17/langchain, 2023. [Accessed 09-Jan-2023].

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Cohen, R., Geva, M., Berant, J., and Globerson, A. Crawling the internal knowledge-base of language models. *arXiv preprint arXiv:2301.12810*, 2023.

Creswell, A. and Shanahan, M. Faithful reasoning using large language models. *arXiv preprint arXiv:2208.14271*, 2022.

Csordás, R., Irie, K., and Schmidhuber, J. The devil is in the detail: Simple tricks improve systematic generalization of transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 619–634, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.49. URL https://aclanthology.org/2021.emnlp-main.49.

Dalvi, B., Tafjord, O., and Clark, P. Towards teachable reasoning systems. *arXiv preprint arXiv:2204.13074*, 2022.

Deletang, G., Ruoss, A., Grau-Moya, J., Genewein, T., Wenliang, L. K., Catt, E., Cundy, C., Hutter, M., Legg, S., Veness, J., and Ortega, P. A. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WbxHAzkeQcn.

Dev, S., Li, T., Phillips, J. M., and Srikumar, V. On measuring and mitigating biased inferences of word embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7659–7666, 2020.

Dohan, D., Xu, W., Lewkowycz, A., Austin, J., Bieber, D., Lopes, R. G., Wu, Y., Michalewski, H., Saurous, R. A., Sohl-Dickstein, J., et al. Language model cascades. *arXiv preprint arXiv:2207.10342*, 2022.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Fan, A., Lavril, T., Grave, E., Joulin, A., and Sukhbaatar, S. Addressing some limitations of transformers with feedback memory, 2021. URL https://openreview.net/forum?id=OCm0rwa1lx1.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022.

Gers, F. A. and Schmidhuber, E. Lstm recurrent networks learn simple context-free and context-sensitive languages. *IEEE transactions on neural networks*, 12(6):1333–1340, 2001.

Geva, M., Khashabi, D., Segal, E., Khot, T., Roth, D., and Berant, J. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9: 346–361, 2021.

Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. *arXiv preprint arXiv:2301.13196*, 2023.

Gupta, P., Jiao, C., Yeh, Y.-T., Mehri, S., Eskenazi, M., and Bigham, J. P. Improving zero and few-shot generalization in dialogue through instruction tuning. *arXiv preprint arXiv:2205.12673*, 2022.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. An empirical analysis of compute-optimal large language model training. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=iBBcRUlOAPR.

Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Jackson, T., Brown, N., Luu, L., Levine, S., Hausman, K., and brian ichter. Inner monologue: Embodied reasoning through planning with language models. In *6th Annual Conference on Robot Learning*, 2022. URL https://openreview.net/forum?id=3R3Pz5i0tye.

Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. Block-recurrent transformers. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=uloenYmLCAo.

Karpas, E., Abend, O., Belinkov, Y., Lenz, B., Lieber, O., Ratner, N., Shoham, Y., Bata, H., Levine, Y., Leyton-Brown, K., et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022.

Kazemi, S. M., Kim, N., Bhatia, D., Xu, X., and Ramachandran, D. Lambada: Backward chaining for automated reasoning in natural language. *arXiv preprint arXiv:2212.13894*, 2022.

Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., and Sabharwal, A. Decomposed prompting: A modular approach for solving complex tasks. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=_nGgzQjzaRy.

Kim, N., Pavlick, E., Ayan, B. K., and Ramachandran, D. Which linguist invented the lightbulb? presupposition verification for question-answering. In *Annual Meeting of the Association for Computational Linguistics*, 2021.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. In *ICML 2022 Workshop on Knowledge Retrieval and Language Models*, 2022. URL https://openreview.net/forum?id=6p3AuaHAFiN.

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., and Petrov, S. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026.

Lazaridou, A., Gribovskaya, E., Stokowiec, W. J., and Grigorev, N. Internet-augmented language models through few-shot prompting for open-domain question answering, 2023. URL https://openreview.net/forum?id=hFCUPkSSRE.

Lee, A., Wu, D., Dinan, E., and Lewis, M. Improving chess commentaries by combining language models with symbolic reasoning engines. *arXiv preprint arXiv:2212.08195*, 2022.

Li, J., Zhang, Z., and Zhao, H. Self-prompting large language models for open-domain qa. *arXiv preprint arXiv:2212.08635*, 2022.

Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A., and Zhang, C. Transformers learn shortcuts to automata. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=De4FYqjFueZ.

Liu, J. GPT Index, 11 2022. URL https://github.com/jerryjliu/gpt_index.

Liu, J., Hallinan, S., Lu, X., He, P., Welleck, S., Hajishirzi, H., and Choi, Y. Rainier: Reinforced knowledge introspector for commonsense question answering. *arXiv preprint arXiv:2210.03078*, 2022a.

Liu, J., Liu, A., Lu, X., Welleck, S., West, P., Le Bras, R., Choi, Y., and Hajishirzi, H. Generated knowledge prompting for commonsense reasoning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3154–3169, Dublin, Ireland, May 2022b. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.

225. URL https://aclanthology.org/2022.acl-long.225.

Medress, M. F., Cooper, F. S., Forgie, J. W., Green, C., Klatt, D. H., O'Malley, M. H., Neuburg, E. P., Newell, A., Reddy, D., Ritea, B., et al. Speech understanding systems: Report of a steering committee. *Artificial Intelligence*, 9 (3):307–316, 1977.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C., and Odena, A. Show your work: Scratchpads for intermediate computation with language models, 2022. URL https://openreview.net/forum?id=iedYJm92o0a.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Gray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=TG8KACxEON.

Paranjape, B., Michael, J., Ghazvininejad, M., Hajishirzi, H., and Zettlemoyer, L. Prompting contrastive explanations for commonsense reasoning tasks. In *Workshop on Commonsense Reasoning and Knowledge Bases*, 2021. URL https://openreview.net/forum?id=KFcjxSNdMBq.

Patel, P., Mishra, S., Parmar, M., and Baral, C. Is a question decomposition unit all we need? *arXiv preprint arXiv:2205.12538*, 2022.

Perez, E., Lewis, P., Yih, W.-t., Cho, K., and Kiela, D. Unsupervised question decomposition for question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8864–8880, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.713. URL https://aclanthology.org/2020.emnlp-main.713.

Reppert, J., Rachbach, B., George, C., Byun, L. S. J., Appleton, M., and Stuhlmüller, A. Iterated decomposition: Improving science q&a by supervising reasoning processes. *arXiv preprint arXiv:2301.01751*, 2023.

Saparov, A. and He, H. Language models can (kind of) reason: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=qFVVBzXxR2V.

Scao, T. L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., Luccioni, A. S., Yvon, F., Gallé, M., et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

Schmidhuber, J. Optimal ordered problem solver. *Machine Learning*, 54(3):211–254, 2004.

Schmidhuber, J. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial general intelligence*, pp. 199–226. Springer, 2007.

Schmidhuber, J. On learning to think: Algorithmic information theory for novel combinations of reinforcement learning controllers and recurrent neural world models. *arXiv preprint arXiv:1511.09249*, 2015.

Schulman, J., Zoph, B., Kim, C., Hilton, J., Menick, J., Weng, J., Ceron Uribe, J. F., Fedus, L., Metz, L., Pokorny, M., Gontijo Lopes, R., Zhao, S., Vijayvergiya, A., Sigler, E., Perelman, A., Voss, C., Heaton, M., Parish, J., Cummings, D., Nayak, R., Balcom, V., Schnurr, D., Kaftan, T., Hallacy, C., Turley, N., Deutsch, N., Goel, V., Ward, J., Konstantinidis, A., Zaremba, W., Ouyang, L., Bogdonoff, L., Gross, J., Medina, D., Yoo, S., Lee, T., Lowe, R., Mossing, D., Huizinga, J., Jiang, R., Wainwright, C., Almeida, D., Lin, S., Zhang, M., Xiao, K., Slama, K., Bills, S., Gray, A., Leike, J., Pachocki, J., Tillet, P., Jain, S., Brockman, G., and Ryder, N., Nov 2022. URL https://openai.com/blog/chatgpt/.

Schuurmans, D. Memory augmented large language models are computationally universal. *arXiv preprint arXiv:2301.04589*, 2023.

Shanahan, M. Talking about large language models. *arXiv preprint arXiv:2212.03551*, 2022.

Sheng, E., Chang, K.-W., Natarajan, P., and Peng, N. Societal biases in language generation: Progress and challenges. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4275–4293, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.330. URL https://aclanthology.org/2021.acl-long.330.

Shuster, K., Komeili, M., Adolphs, L., Roller, S., Szlam, A., and Weston, J. Language models that seek for knowledge: Modular search & generation for dialogue and prompt completion. *arXiv preprint arXiv:2203.13224*, 2022a.

Shuster, K., Xu, J., Komeili, M., Ju, D., Smith, E. M., Roller, S., Ung, M., Chen, M., Arora, K., Lane, J., et al. Blenderbot 3: a deployed conversational agent that continually learns to responsibly engage. *arXiv preprint arXiv:2208.03188*, 2022b.

Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.

Taylor, R., Kardas, M., Cucurull, G., Scialom, T., Hartshorn, A., Saravia, E., Poulton, A., Kerkez, V., and Stojnic, R. Galactica: A large language model for science. *arXiv preprint arXiv:2211.09085*, 2022.

Thoppilan, R., De Freitas, D., Hall, J., Shazeer, N., Kulshreshtha, A., Cheng, H.-T., Jin, A., Bos, T., Baker, L., Du, Y., et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.

Valmeekam, K., Olmo, A., Sreedharan, S., and Kambhampati, S. Large language models still can't plan (a benchmark for LLMs on planning and reasoning about change). In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022. URL https://openreview.net/forum?id=wUU-7XTL5XO.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Proc. Advances in Neural Information Processing Systems (NIPS)*, pp. 5998–6008, Long Beach, CA, USA, December 2017.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.

Wang, Y., Mishra, S., Alipoormolabashi, P., Kordi, Y., Mirzaei, A., Naik, A., Ashok, A., Dhanasekaran, A. S., Arunkumar, A., Stap, D., Pathak, E., Karamanolakis, G., Lai, H., Purohit, I., Mondal, I., Anderson, J., Kuznia, K., Doshi, K., Pal, K. K., Patel, M., Moradshahi, M., Parmar, M., Purohit, M., Varshney, N., Kaza, P. R., Verma, P., Puri, R. S., Karia, R., Doshi, S., Sampat, S. K., Mishra, S., Reddy A, S., Patro, S., Dixit, T., and Shen, X. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 5085–5109, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.emnlp-main.340.

Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022a. URL https://openreview.net/forum?id=gEZrGCozdqR.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., brian ichter, Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain of thought prompting elicits reasoning in large language models. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022b. URL https://openreview.net/forum?id=_VjQlMeSB_J.

Wu, J., Ouyang, L., Ziegler, D. M., Stiennon, N., Lowe, R., Leike, J., and Christiano, P. F. Recursively summarizing books with human feedback. *CoRR*, abs/2109.10862, 2021a. URL https://arxiv.org/abs/2109.10862.

Wu, J., Ouyang, L., Ziegler, D. M., Stiennon, N., Lowe, R., Leike, J., and Christiano, P. F. Recursively summarizing books with human feedback. *CoRR*, abs/2109.10862, 2021b. URL https://arxiv.org/abs/2109.10862.

Wu, T., Terry, M., and Cai, C. J. Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pp. 1–22, 2022a.

Wu, T. S., Jiang, E., Donsbach, A., Gray, J., Molina, A., Terry, M., and Cai, C. J. Promptchainer: Chaining large language model prompts through visual programming. *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, 2022b.

Xu, J., Szlam, A. D., and Weston, J. Beyond goldfish memory: Long-term open-domain conversation. In *Annual Meeting of the Association for Computational Linguistics*, 2021.

Yang, J., Jiang, H., Yin, Q., Zhang, D., Yin, B., and Yang, D. SEQZERO: Few-shot compositional semantic parsing with sequential prompts and zero-shot models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pp. 49–60, Seattle, United States, July 2022a. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-naacl.5. URL https://aclanthology.org/2022.findings-naacl.5.

Yang, K., Peng, N., Tian, Y., and Klein, D. Re3: Generating longer stories with recursive reprompting and revision. *arXiv preprint arXiv:2210.06774*, 2022b.

Yang, Z., Dong, L., Du, X., Cheng, H., Cambria, E., Liu, X., Gao, J., and Wei, F. Language models as inductive reasoners. *arXiv preprint arXiv:2212.10923*, 2022c.

Yao, S., Chen, H., Yang, J., and Narasimhan, K. R. Webshop: Towards scalable real-world web interaction with grounded language agents. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=R9KnuFlvnU.

Zelikman, E., Wu, Y., Mu, J., and Goodman, N. STar: Bootstrapping reasoning with reasoning. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=_3ELRdg2sgI.

Zeng, A., Attarian, M., brian ichter, Choromanski, K. M., Wong, A., Welker, S., Tombari, F., Purohit, A., Ryoo, M. S., Sindhwani, V., Lee, J., Vanhoucke, V., and Florence, P. Socratic models: Composing zero-shot multimodal reasoning with language. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=G2Q2Mh3avow.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q. V., and Chi, E. H. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023a. URL https://openreview.net/forum?id=WZH7099tgfM.

Zhou, H., Nova, A., Courville, A., Larochelle, H., Neyshabur, B., and Sedghi, H. Teaching algorithmic reasoning via in-context learning, 2023b. URL https://openreview.net/forum?id=6dlC7E1H_9.

A. Algorithms

A.1. Ranking Algorithm

Algorithm 1 Ranking paragraphs by the average negative log-likelihood of the question and returning the top-n.

```
Input: question q, paragraphs P, n
initialise: nlls = [ ]
for paragraph p in P do
    avg_nll = LLM(q, p)
    nlls.append((i, avg_nll))
end for
return sorted(nlls)[:n]
```

A.2. Tree Search Algorithm

Algorithm 2 Tree search over steps generated conditioned on different evidence paragraphs ranked by a ranking criteria r.

```
Input: question q, paragraphs P, criteria r
initialise: chains = [[q]], complete = [ ]
while len(complete) <= 3 do
    c = pop(sort(chains, criteria =r))
    for paragraph p in P do
        c' = add_step(c, p)
        if gives_answer(c') then
            complete.append(c')
        else
            chains.append(c')
        end if
    end for
end while
return complete
```